מדריך מהיר ל-Claude Code

מההתקנה ועד עבודה כמו מקצוען - בלי הטעויות שעולות ביוקר בכסף, בזמן ובעבודה.

מה בפנים?

- התקנה נכונה ב-30 שניות
- 5 הטעויות שעולות ביוקר (ואיך לעקוף אותן)
- הוורקפלואו של מקצוען, צעד אחרי צעד
- Cheat-sheet עם כל הפקודות החשובות
- תבנית CLAUDE.md מוכנה להעתקה

רגע, מה זה Claude Code?

זה לא עוד צ'אט. **Claude Code הוא סוכן AI שחי בטרמינל, בתוך הפרויקט שלכם.** הוא קורא את הקבצים שלכם, כותב קוד, מריץ פקודות ומבצע משימות שלמות בעצמו. אתם מנהלים אותו במילים פשוטות, והוא עושה את העבודה.

ההתקנה היא החלק הקל

מתקינים בשורה אחת בטרמינל. זו דרך ההתקנה הרשמית המומלצת היום (npm עדיין נתמך, כחלופה):

```
curl -fsSL https://claude.ai/install.sh | bash
```

אחרי ההתקנה מריצים claude ומתחברים לחשבון. **שימו לב:** לעבודה צריך מנוי כמו Pro או Max (החל מ-20 דולר לחודש) - הגרסה החינמית לא כוללת את Claude Code.

נתקעתם? אם מופיע command not found אחרי ההתקנה, לא נכשלתם - רק צריך להוסיף שורה אחת ל-PATH, לפתוח טרמינל מחדש, וזה עובד:

```
export PATH="$HOME/.local/bin:$PATH"
```

5 הטעויות של מתחילים (ואיך לעקוף אותן)

לכל טעות: מה שמתחיל עושה, ומה שמקצוען עושה במקום.

01. משתמשים בו כמו צ'אט

מתחיל: שואל שאלה, מקבל תשובה, ומעתיק את הקוד ביד.

מקצוען: "תתקן את הבאג ב-checkout ותכתוב טסט." הוא קורא, כותב ומריץ בעצמו.

02. עובדים בלי Git

מתחיל: נותן ל-Claude לשנות ומקווה לטוב. נשבר? כל העבודה הלכה.

מקצוען: עושה commit לפני כל שינוי גדול. נשבר? פקודה אחת וחוזרים למצב הקודם.

03. שורפים את המנוי

מתחיל: עובד שעות באותה שיחה ענקית עד שהכל איטי, יקר ומבולבל.

מקצוען: מפרק משימות לתתי-משימות, ופותח שיחות חדשות כדי לא לאבד הקשר.

טיפ: /clear מתחיל שיחה נקייה · /compact מצמצם ושומר את העיקר · /context מראה כמה הקשר נשאר.

04. תמיד על המודל הכי חזק

המודל החזק (Opus) מעולה למשימות מורכבות, אבל על דברים קטנים זה בזבוז - הוא יקר פי 5 מהמודל המהיר. התאימו את המודל למשימה.

✗	**מתחיל:** משאיר את Opus על הכל, גם על תיקון של פסיק.
✓	**מקצוען:** /model haiku לקטן · sonnet ליומיומי · opus רק לקשה.

טיפ: למשימות גדולות, תכננו עם Opus ובצעו עם Sonnet - תוצאות מעולות במינימום טוקנים.

05. עובדים ללא CLAUDE.md

בלי קובץ הוראות קבוע, Claude שוכח את הפרויקט בכל פעם מחדש, ואתם מבזבזים חצי שיחה רק כדי להסביר מאיפה להתחיל.

✗	**מתחיל:** כותב שוב ושוב "תכתוב בעברית, אל תיגע בקובץ הזה".
✓	**מקצוען:** מריץ /init פעם אחת. הקובץ נטען בכל שיחה והוא זוכר את ההקשר.

טיפ: הקובץ מוזרק לזיכרון בכל שיחה. כתבו בו רק את הדברים החשובים ביותר, ומומלץ לשמור אותו מתחת ל-200 שורות.

הוורקפלואו של מקצוען - 6 צעדים

תפסתם את הטעויות? עכשיו תהפכו אותן לשגרת עבודה אחת שעובדת בכל פעם.

1. **פותחים עם CLAUDE.md.** מריצים `init/` בפעם הראשונה בפרויקט. הקובץ מזכיר ל-Claude את הכללים שלכם בכל שיחה.
2. **מתכננים לפני שבונים.** במשימה מורכבת, בקשו קודם תוכנית (plan mode). אפשר לתכנן עם Opus ולבצע עם Sonnet.
3. **נותנים לו לעשות, ובודקים.** נסחו משימה ממוקדת, תנו ל-Claude לקרוא, לכתוב ולהריץ - ובדקו את ה-diff לפני שמאשרים.
4. **עושים commit תכוף.** לפני כל שינוי גדול. Git הוא כפתור החזרה אחורה שלכם.
5. **שומרים על שיחה ממוקדת.** `clear/` בין משימות, `compact/` כשהשיחה מתארכת.
6. **בוחרים מודל לפי המשימה.** Haiku לקטן, Sonnet ליומיומי, Opus לקשה באמת.

הפקודות שצריך לדעת

שמרו את העמוד הזה. אלה הפקודות שתשתמשו בהן כל יום.

`/init`	יוצר קובץ CLAUDE.md לפרויקט (הוראות קבועות).
`/clear`	מתחיל שיחה נקייה. השיחה הקודמת נשמרת.
`/compact`	מצמצם את השיחה ושומר רק את הנקודות החשובות.
`/context`	מראה כמה מההקשר (context) כבר בשימוש.
`/model haiku|sonnet|opus`	מחליף מודל לפי גודל המשימה.
`Esc Esc`	חזרה אחורה (rewind) לנקודה קודמת בשיחה.
`Shift+Tab ×2`	מצב תכנון (plan mode) - חוקר ומציע בלי לשנות קבצים.

Git - הבסיס שמציל עבודה

`git init`	הופך את התיקייה למעקב גרסאות.
`git add -A && git commit -m "..."`	שומר נקודת שמירה של כל השינויים.
`git reset --hard HEAD`	חוזר לנקודת השמירה האחרונה (מבטל שינויים).

תבנית CLAUDE.md

העתיקו, מלאו, ושמרו בשורש הפרויקט. זה ה-ROI הכי גבוה שתעשו.

```
# הפרויקט שלי

## מה זה
<תיאור קצר של מה הפרויקט עושה>

## פקודות
- build: <פקודת הבנייה>
- test:  <פקודת הבדיקות>
- run:   <פקודת ההרצה>

## כללים
- ענה והערות בעברית
- אל תיגע בקבצים: .env, <קבצים רגישים>
- סגנון: <קונבנציות הקוד שלי>

## מבנה
- קוד עיקרי: src/
- בדיקות:    test/
```

Checklist לפני כל סשן

1. נכנסתי לתיקיית הפרויקט הנכונה (cd)
2. יש git בפרויקט (אם לא: git init)
3. קובץ CLAUDE.md קיים ומעודכן
4. בחרתי מודל שמתאים לגודל המשימה
5. ניסחתי משימה ממוקדת וברורה (לא "תעשה הכל")

השיחה הראשונה צעד אחרי צעד

ככה נראית שיחת Claude Code ראשונה, מההתחלה עד תוצאה.

1. **נכנסים לפרויקט ומפעילים.** `cd my-project` **ואז** `claude`.
2. **נותנים לו להבין לבד.** "מה הפרויקט הזה עושה?" - הוא קורא את הקבצים ומסביר.
3. **מבקשים משימה ממוקדת.** "תוסיף כפתור שמירה לטופס, ותכתוב טסט קודם."
4. **בודקים ומאשרים.** עוברים על ה-diff שהוא מציע, ומאשרים אם זה טוב.
5. **שומרים.** "תעשה commit עם הודעה ברורה." וזהו - יש לכם נקודת שמירה.

רוצים להפוך למקצוענים?

זה רק ההתחלה. עוקבים אחרי **@vaknin.ai** לסדרת ה-Claude המלאה - כלים, פרומפטים ואוטומציות בעברית, ליישום מיידי.

אני גל, מקים **vaknin.ai**. יש שאלה? כתבו לי ב-DM. 🤖